```
from datetime import date, datetime
import hashlib
import hmac
import json
from pathlib import Path
import secrets
import tkinter as tk
from tkinter import messagebox, ttk
import uuid


DAILY_WITHDRAWAL_LIMIT = 20000
PIN_HASH_ITERATIONS = 120000


class ATMApp:
    def __init__(self, root):
        self.root = root
        self.root.title("ATM Simulation System")
        self.root.geometry("600x760")
        self.root.minsize(540, 650)
        self.root.configure(bg="#f2f5f9")

        self.data_path = Path(__file__).with_name("atm_data.json")
        self.accounts = {}
        self.transactions = []
        self.cheque_numbers = set()
```

```
        self.account = None
        self.colors = {
            "navy": "#17324d",
            "blue": "#2364aa",
            "background": "#f2f5f9",
            "white": "#ffffff",
            "text": "#1f2937",
            "muted": "#64748b",
        }

        self.style = ttk.Style()
        self.style.configure("ATM.TButton", padding=(12, 8), font=("Segoe UI", 10))
        self.style.configure(
            "ATMPrimary.TButton",
            padding=(12, 9),
            font=("Segoe UI", 10, "bold"),
        )
        self.style.configure("ATM.TEntry", padding=8)

        if self.load_data():
            self.show_home()

    def load_data(self):
        if not self.data_path.exists():
            return True
        try:
11
            with self.data_path.open("r", encoding="utf-8") as data_file:
```

```
            data = json.load(data_file)
        self.accounts = data.get("accounts", {})
        self.transactions = data.get("transactions", [])
        if not isinstance(self.accounts, dict) or not isinstance(self.transactions,
list):
            raise ValueError("The saved ATM data has an unexpected format.")
        self.cheque_numbers = {
            item.get("cheque_number")
            for item in self.transactions
            if item.get("cheque_number")
        }
        return True
    except (OSError, json.JSONDecodeError, ValueError, AttributeError) as error:
        messagebox.showerror(
            "Could not load saved data",
            f"The ATM data file could not be read:\n{error}\n\n"
            "The app will close so the saved data is not overwritten.",
            parent=self.root,
        )
        self.root.destroy()
        return False

  def save_data(self):
    data = {"accounts": self.accounts, "transactions": self.transactions}
    temporary_path = self.data_path.with_suffix(".tmp")
    try:
        with temporary_path.open("w", encoding="utf-8") as data_file:
            json.dump(data, data_file, indent=2)
```

```
            temporary_path.replace(self.data_path)
            return True
        except OSError as error:
            messagebox.showerror(
                "Could not save data",
                f"The ATM data could not be saved:\n{error}",
                parent=self.root,
            )
            return False


    @staticmethod
    def make_pin_hash(pin, salt):
        return hashlib.pbkdf2_hmac(
            "sha256",
            pin.encode("utf-8"),
            bytes.fromhex(salt),
            PIN_HASH_ITERATIONS,
        ).hex()


    def pin_matches(self, account, pin):
        try:
            candidate = self.make_pin_hash(pin, account["pin_salt"])
            return hmac.compare_digest(candidate, account["pin_hash"])
        except (KeyError, ValueError):
            return False


    def clear_screen(self):
        for child in self.root.winfo_children():
```

```
        child.destroy()


12
    def make_header(self, subtitle):
        header = tk.Frame(self.root, bg=self.colors["navy"], padx=28, pady=20)
        header.pack(fill="x")
        tk.Label(
            header,
            text="ATM Simulation System",
            font=("Segoe UI", 19, "bold"),
            fg="white",
            bg=self.colors["navy"],
        ).pack(anchor="w")
        tk.Label(
            header,
            text=subtitle,
            font=("Segoe UI", 10),
            fg="#d6e2f0",
            bg=self.colors["navy"],
        ).pack(anchor="w", pady=(5, 0))

    def make_card(self):
        card = tk.Frame(self.root, bg=self.colors["white"], padx=28, pady=24)
        card.pack(fill="both", expand=True, padx=30, pady=24)
        return card

    def add_field(self, parent, label, show=None):
        tk.Label(
```

```
        parent,
        text=label,
        font=("Segoe UI", 10, "bold"),
        fg=self.colors["text"],
        bg=self.colors["white"],
    ).pack(anchor="w", pady=(12, 5))
    entry = ttk.Entry(parent, style="ATM.TEntry", show=show or "")
    entry.pack(fill="x")
    return entry

def show_home(self):
    self.clear_screen()
    self.make_header("Create an account or log in to continue")
    card = self.make_card()
    tk.Label(
        card,
        text="Welcome",
        font=("Segoe UI", 18, "bold"),
        fg=self.colors["text"],
        bg=self.colors["white"],
    ).pack(anchor="w")
    tk.Label(
        card,
        text="Your accounts and transaction history are saved on this computer.",
        font=("Segoe UI", 10),
        fg=self.colors["muted"],
        bg=self.colors["white"],
        wraplength=450,
```

```
            justify="left",
        ).pack(anchor="w", pady=(6, 24))
        ttk.Button(
            card,
            text="Log in to an account",
            style="ATMPrimary.TButton",
13
            command=self.show_login,
        ).pack(fill="x", pady=5)
        ttk.Button(
            card,
            text="Create a new account",
            style="ATM.TButton",
            command=self.show_create_account,
        ).pack(fill="x", pady=5)
        ttk.Button(
            card,
            text="Close",
            style="ATM.TButton",
            command=self.root.destroy,
        ).pack(fill="x", pady=(18, 0))

    def show_create_account(self):
        self.clear_screen()
        self.make_header("Add another account to this ATM simulation")
        card = self.make_card()
        tk.Label(
            card,
```

```
            text="Create Account",
            font=("Segoe UI", 16, "bold"),
            fg=self.colors["text"],
            bg=self.colors["white"],
        ).pack(anchor="w")

        self.name_entry = self.add_field(card, "Account holder name")
        self.account_entry = self.add_field(card, "Account number (at least 6 digits)")
        self.pin_entry = self.add_field(card, "Create a 4-digit ATM PIN", show="*")
        self.balance_entry = self.add_field(card, "Opening balance (Rs.)")
        ttk.Button(
            card,
            text="Create account",
            style="ATMPrimary.TButton",
            command=self.create_account,
        ).pack(fill="x", pady=(22, 5))
        ttk.Button(card, text="Cancel", command=self.show_home).pack(fill="x", pady=4)

    def create_account(self):
        name = self.name_entry.get().strip()
        account_number = self.account_entry.get().strip()
        pin = self.pin_entry.get().strip()
        balance_text = self.balance_entry.get().strip()

        if not name or not all(part.isalpha() for part in name.split()):
            messagebox.showerror("Invalid name", "Enter a name using letters only.",
parent=self.root)
            return
```

```
        if not account_number.isdigit() or len(account_number) < 6:
            messagebox.showerror(
                "Invalid account number",
                "The account number must contain at least 6 digits.",
                parent=self.root,
            )
            return
        if account_number in self.accounts:
            messagebox.showerror(
                "Account already exists",
14
                "Choose a different account number.",
                parent=self.root,
            )
            return
        if len(pin) != 4 or not pin.isdigit():
            messagebox.showerror("Invalid PIN", "The PIN must contain exactly 4 dig
its.", parent=self.root)
            return
        try:
            balance = float(balance_text)
        except ValueError:
            messagebox.showerror("Invalid balance", "Enter a valid opening balance.",
parent=self.root)
            return
        if balance < 0 or not balance.is_integer():
            messagebox.showerror(
                "Invalid balance",
```

```
            "The opening balance must be a non-negative whole amount.",
            parent=self.root,
        )
        return

    salt = secrets.token_hex(16)
    self.accounts[account_number] = {
        "name": name,
        "account_number": account_number,
        "pin_salt": salt,
        "pin_hash": self.make_pin_hash(pin, salt),
        "balance": balance,
        "failed_attempts": 0,
        "locked": False,
    }
    if self.save_data():
        messagebox.showinfo("Account created", "Your account was created success
fully.", parent=self.root)
        self.show_login()

  def show_login(self):
    self.clear_screen()
    self.make_header("Enter your account number and PIN")
    card = self.make_card()
    tk.Label(
        card,
        text="Account Login",
        font=("Segoe UI", 16, "bold"),
```

```
            fg=self.colors["text"],
            bg=self.colors["white"],
        ).pack(anchor="w")
        self.login_account_entry = self.add_field(card, "Account number")
        self.login_pin_entry = self.add_field(card, "ATM PIN", show="*")
        ttk.Button(
            card,
            text="Log in",
            style="ATMPrimary.TButton",
            command=self.login,
        ).pack(fill="x", pady=(22, 5))
        ttk.Button(
            card,
            text="Create an account",
15
            command=self.show_create_account,
        ).pack(fill="x", pady=4)
        ttk.Button(card, text="Cancel", command=self.show_home).pack(fill="x", pady=4)

    def login(self):
        account_number = self.login_account_entry.get().strip()
        pin = self.login_pin_entry.get().strip()
        account = self.accounts.get(account_number)
        if account is None:
            messagebox.showerror("Login failed", "Account number was not found.", par
ent=self.root)
            return
        if account.get("locked", False):
```

```
            messagebox.showerror(
                "Account locked",
                "This account is locked after three incorrect PIN attempts. Contact the
ATM administrator.",
                parent=self.root,
            )
            return
        if not self.pin_matches(account, pin):
            account["failed_attempts"] = account.get("failed_attempts", 0) + 1
            attempts_left = max(0, 3 - account["failed_attempts"])
            if attempts_left == 0:
                account["locked"] = True
                message = "Three incorrect PIN attempts. This account is now locked."
            else:
                message = f"Incorrect PIN. You have {attempts_left} attempt(s) remain
ing."
            self.save_data()
            messagebox.showerror("Login failed", message, parent=self.root)
            return

        account["failed_attempts"] = 0
        if not self.save_data():
            return
        self.account = account
        self.show_atm_menu()

    def ask_fields(self, title, fields):
        dialog = tk.Toplevel(self.root)
```

```
    dialog.title(title)
    dialog.configure(bg=self.colors["white"])
    dialog.transient(self.root)
    dialog.resizable(False, False)
    dialog.grab_set()

    frame = tk.Frame(dialog, bg=self.colors["white"], padx=22, pady=18)
    frame.pack(fill="both", expand=True)
    entries = []
    for label, secret in fields:
        tk.Label(
            frame,
            text=label,
            font=("Segoe UI", 10, "bold"),
            fg=self.colors["text"],
            bg=self.colors["white"],
        ).pack(anchor="w", pady=(9, 4))
        entry = ttk.Entry(frame, style="ATM.TEntry", show="*" if secret else "")
16
        entry.pack(fill="x")
        entries.append(entry)

    result = {"values": None}

    def submit():
        result["values"] = [entry.get().strip() for entry in entries]
        dialog.destroy()
```

```
        def cancel():
            result["values"] = None
            dialog.destroy()

        buttons = tk.Frame(frame, bg=self.colors["white"])
        buttons.pack(fill="x", pady=(18, 0))
        ttk.Button(buttons, text="Submit", style="ATMPrimary.TButton", command=sub
mit).pack(
            side="left", expand=True, fill="x", padx=(0, 5)
        )
        ttk.Button(buttons, text="Cancel", command=cancel).pack(
            side="left", expand=True, fill="x", padx=(5, 0)
        )
        dialog.protocol("WM_DELETE_WINDOW", cancel)
        if entries:
            entries[0].focus_set()
        dialog.bind("<Return>", lambda _event: submit())
        self.root.wait_window(dialog)
        return result["values"]

    @staticmethod
    def now_string():
        return datetime.now().astimezone().isoformat(timespec="seconds")

    @staticmethod
    def display_time(value):
        try:
            return datetime.fromisoformat(value).astimezone().strftime("%d-%m-%Y
```

```
%H:%M:%S")
        except (TypeError, ValueError):
            return str(value)

    def record_transaction(
        self,
        account_number,
        transaction_type,
        amount,
        balance,
        status="Completed",
        reference=None,
        cheque_number=None,
        recipient=None,
    ):
        self.transactions.append(
            {
                "date": self.now_string(),
                "account": account_number,
                "type": transaction_type,
                "amount": amount,
                "balance": balance,
```



```
                "status": status,
                "reference": reference or uuid.uuid4().hex[:10].upper(),
                "cheque_number": cheque_number,
                "recipient": recipient,
            }
```

```
    )
    return self.transactions[-1]

def pending_cheque_total(self, account_number):
    return sum(
        float(item["amount"])
        for item in self.transactions
        if item.get("account") == account_number
        and item.get("type") == "Cheque Deposit"
        and item.get("status") == "Pending clearance"
    )

def show_atm_menu(self):
    self.clear_screen()
    pending = self.pending_cheque_total(self.account["account_number"])
    self.make_header(f"Welcome, {self.account['name']}")
    card = self.make_card()
    tk.Label(
        card,
        text="ATM Main Menu",
        font=("Segoe UI", 16, "bold"),
        fg=self.colors["text"],
        bg=self.colors["white"],
    ).pack(anchor="w")
    tk.Label(
        card,
        text=(
            f"Account ending in {self.account['account_number'][-4:]} • "
```

```
            f"Available: Rs. {self.account['balance']:.2f} • "
            f"Pending cheques: Rs. {pending:.2f}"
        ),
        font=("Segoe UI", 9),
        fg=self.colors["muted"],
        bg=self.colors["white"],
        wraplength=470,
        justify="left",
    ).pack(anchor="w", pady=(5, 12))

    actions = [
        ("Balance enquiry", self.balance_enquiry),
        ("Cash deposit", self.deposit_money),
        ("Cheque deposit", self.deposit_cheque),
        ("Cash withdrawal", self.withdraw_money),
        ("Transfer to another account", self.transfer_money),
        ("Clear pending cheque deposits", self.clear_pending_cheques),
        ("Change ATM PIN", self.change_pin),
        ("Mini statement", self.mini_statement),
    ]
    for label, command in actions:
        ttk.Button(card, text=label, style="ATM.TButton", command=com
mand).pack(fill="x", pady=2)
    ttk.Button(card, text="Log out", command=self.logout).pack(fill="x", pady=(10,
0))
```



```
  def refresh_menu(self):
```

```
        if self.account is not None:
            self.show_atm_menu()

    def parse_amount(self, value, label="amount"):
        try:
            amount = float(value)
        except ValueError:
            messagebox.showerror("Invalid amount", f"Enter a valid {label}.", par
ent=self.root)
            return None
        if amount <= 0:
            messagebox.showerror("Invalid amount", "The amount must be greater than
zero.", parent=self.root)
            return None
        if not amount.is_integer():
            messagebox.showerror("Invalid amount", "Enter a whole amount in rupees.",
parent=self.root)
            return None
        return amount

    def show_receipt(self, title, lines):
        receipt = tk.Toplevel(self.root)
        receipt.title(title)
        receipt.geometry("450x470")
        receipt.minsize(390, 360)
        receipt.configure(bg=self.colors["white"])
        receipt.transient(self.root)
```

```
        tk.Label(
            receipt,
            text="ATM SIMULATION RECEIPT",
            font=("Segoe UI", 13, "bold"),
            fg=self.colors["navy"],
            bg=self.colors["white"],
        ).pack(anchor="w", padx=20, pady=(18, 8))
        text_area = tk.Text(
            receipt,
            wrap="word",
            height=18,
            font=("Consolas", 10),
            bg="#f8fafc",
            relief="flat",
            padx=12,
            pady=12,
        )
        text_area.pack(fill="both", expand=True, padx=20, pady=8)
        text_area.insert("1.0", "\n".join(lines))
        text_area.configure(state="disabled")
        ttk.Button(receipt, text="Close", command=receipt.destroy).pack(fill="x",
padx=20, pady=(4, 18))

    def receipt_lines(self, transaction):
        lines = [
            f"Date:     {self.display_time(transaction['date'])}",
            f"Type:     {transaction['type']}",
            f"Account:  {transaction['account']}",
```

```
            f"Amount:    Rs. {float(transaction['amount']):.2f}",
            f"Status:    {transaction.get('status', 'Completed')}",
19
            f"Reference: {transaction.get('reference', 'N/A')}",
        ]
        if transaction.get("recipient"):
            lines.append(f"Recipient: {transaction['recipient']}")
        if transaction.get("cheque_number"):
            lines.append(f"Cheque no: {transaction['cheque_number']}")
        lines.append(f"Balance:   Rs. {float(transaction['balance']):.2f}")
        return lines

    def balance_enquiry(self):
        pending = self.pending_cheque_total(self.account["account_number"])
        messagebox.showinfo(
            "Balance Enquiry",
            f"Account holder: {self.account['name']}\n"
            f"Account number: {self.account['account_number']}\n"
            f"Available balance: Rs. {self.account['balance']:.2f}\n"
            f"Pending cheque deposits: Rs. {pending:.2f}",
            parent=self.root,
        )

    def deposit_money(self):
        values = self.ask_fields("Cash Deposit", [("Deposit amount (Rs.)", False)])
        if values is None:
            return
        amount = self.parse_amount(values[0])
```

```
        if amount is None:
            return
        self.account["balance"] += amount
        transaction = self.record_transaction(
            self.account["account_number"], "Cash Deposit", amount, self.account["bal
ance"]
        )
        if self.save_data():
            self.refresh_menu()
            self.show_receipt("Cash Deposit Receipt", self.receipt_lines(transaction))

    def deposit_cheque(self):
        values = self.ask_fields(
            "Cheque Deposit",
            [("Cheque number (6 to 12 digits)", False), ("Cheque amount (Rs.)", False)],
        )
        if values is None:
            return
        cheque_number = values[0]
        if not cheque_number.isdigit() or not 6 <= len(cheque_number) <= 12:
            messagebox.showerror(
                "Invalid cheque number",
                "The cheque number must contain 6 to 12 digits.",
                parent=self.root,
            )
            return
        if cheque_number in self.cheque_numbers:
            messagebox.showerror(
```

```
            "Cheque already deposited",
            "This cheque number has already been used.",
            parent=self.root,
        )
        return
    amount = self.parse_amount(values[1], "cheque amount")
20
    if amount is None:
        return

    self.cheque_numbers.add(cheque_number)
    transaction = self.record_transaction(
        self.account["account_number"],
        "Cheque Deposit",
        amount,
        self.account["balance"],
        status="Pending clearance",
        cheque_number=cheque_number,
    )
    if self.save_data():
        self.refresh_menu()
        lines = self.receipt_lines(transaction)
        lines.extend(["", "The amount is not available until the cheque is
cleared."])
        self.show_receipt("Cheque Deposit Receipt", lines)

  def daily_withdrawn(self, account_number):
    today = date.today()
```

```
        total = 0.0
        for item in self.transactions:
            if item.get("account") != account_number or item.get("type") != "Cash With
drawal":
                continue
            try:
                if datetime.fromisoformat(item["date"]).astimezone().date() == today:
                    total += float(item["amount"])
            except (KeyError, TypeError, ValueError):
                continue
        return total


    def withdraw_money(self):
        values = self.ask_fields("Cash Withdrawal", [("Withdrawal amount (Rs.)",
False)])
        if values is None:
            return
        amount = self.parse_amount(values[0], "withdrawal amount")
        if amount is None:
            return
        if amount % 100 != 0:
            messagebox.showerror(
                "Invalid withdrawal",
                "Withdrawals must be in multiples of Rs. 100.",
                parent=self.root,
            )
            return
        remaining_limit = DAILY_WITHDRAWAL_LIMIT - self.daily_withdrawn(self.ac
```

```
count["account_number"])
        if amount > remaining_limit:
            messagebox.showerror(
                "Daily limit exceeded",
                f"The daily withdrawal limit is Rs. {DAILY_WITHDRAWAL_LIMIT:.2f}.\n"
                f"You can withdraw up to Rs. {max(0, remaining_limit):.2f} more today.",
                parent=self.root,
            )
            return
        if amount > self.account["balance"]:
21
            messagebox.showerror(
                "Insufficient balance",
                "There is not enough money in the account.",
                parent=self.root,
            )
            return

        self.account["balance"] -= amount
        transaction = self.record_transaction(
            self.account["account_number"], "Cash Withdrawal", amount, self.ac
count["balance"]
        )
        if self.save_data():
            self.refresh_menu()
            self.show_receipt("Withdrawal Receipt", self.receipt_lines(transaction))

    def transfer_money(self):
```

```
        values = self.ask_fields(
            "Transfer to Another Account",
            [("Recipient account number", False), ("Transfer amount (Rs.)", False)],
        )
        if values is None:
            return
        recipient_number = values[0]
        if not recipient_number.isdigit() or len(recipient_number) < 6:
            messagebox.showerror("Invalid account number", "Enter a valid account num
ber.", parent=self.root)
            return
        if recipient_number == self.account["account_number"]:
            messagebox.showerror("Invalid transfer", "You cannot transfer to the same
account.", parent=self.root)
            return
        recipient = self.accounts.get(recipient_number)
        if recipient is None:
            messagebox.showerror("Account not found", "The recipient account does not
exist.", parent=self.root)
            return
        if recipient.get("locked", False):
            messagebox.showerror("Account locked", "The recipient account is locked.",
parent=self.root)
            return
        amount = self.parse_amount(values[1], "transfer amount")
        if amount is None:
            return
        if amount > self.account["balance"]:
```

```
            messagebox.showerror(
                "Insufficient balance",
                "There is not enough money in the account.",
                parent=self.root,
            )
            return

        reference = uuid.uuid4().hex[:10].upper()
        self.account["balance"] -= amount
        recipient["balance"] += amount
        transaction = self.record_transaction(
            self.account["account_number"],
            "Transfer Sent",
            amount,
22
            self.account["balance"],
            reference=reference,
            recipient=recipient_number,
        )
        self.record_transaction(
            recipient_number,
            "Transfer Received",
            amount,
            recipient["balance"],
            reference=reference,
            recipient=self.account["account_number"],
        )
        if self.save_data():
```

```
        self.refresh_menu()
        self.show_receipt("Transfer Receipt", self.receipt_lines(transaction))

    def clear_pending_cheques(self):
        pending = [
            item
            for item in self.transactions
            if item.get("account") == self.account["account_number"]
            and item.get("type") == "Cheque Deposit"
            and item.get("status") == "Pending clearance"
        ]
        if not pending:
            messagebox.showinfo("Cheque Clearance", "There are no pending cheques to clear.", parent=self.root)
            return

        total = 0.0
        cheque_list = []
        for item in pending:
            amount = float(item["amount"])
            total += amount
            self.account["balance"] += amount
            item["status"] = "Cleared"
            cheque_list.append(item.get("cheque_number", "Unknown"))
            self.record_transaction(
                self.account["account_number"],
                "Cheque Clearance",
                amount,
```

```
            self.account["balance"],
            reference=item.get("reference"),
            cheque_number=item.get("cheque_number"),
        )
    if self.save_data():
        self.refresh_menu()
        self.show_receipt(
            "Cheque Clearance Receipt",
            [
                f"Date: {datetime.now().astimezone().strftime('%d-%m-%Y
%H:%M:%S')}",
                "Status: Cheques cleared",
                "Cheque number(s): " + ", ".join(cheque_list),
                f"Total credited: Rs. {total:.2f}",
                f"Available balance: Rs. {self.account['balance']:.2f}",
            ],
        )
```

23

```
def change_pin(self):
    values = self.ask_fields(
        "Change ATM PIN",
        [("Current PIN", True), ("New 4-digit PIN", True), ("Confirm new PIN",
True)],
    )
    if values is None:
        return
    old_pin, new_pin, confirm = values
```

```
        if not self.pin_matches(self.account, old_pin):
            messagebox.showerror("Incorrect PIN", "The current PIN is incorrect.", par
ent=self.root)
            return
        if len(new_pin) != 4 or not new_pin.isdigit():
            messagebox.showerror("Invalid PIN", "The PIN must contain exactly 4 dig
its.", parent=self.root)
            return
        if new_pin == old_pin:
            messagebox.showerror("Invalid PIN", "The new PIN must differ from the cur
rent PIN.", parent=self.root)
            return
        if confirm != new_pin:
            messagebox.showerror("PIN mismatch", "The PINs do not match.", par
ent=self.root)
            return

        salt = secrets.token_hex(16)
        self.account["pin_salt"] = salt
        self.account["pin_hash"] = self.make_pin_hash(new_pin, salt)
        if self.save_data():
            messagebox.showinfo("PIN changed", "Your ATM PIN was changed successfully.",
parent=self.root)

    def mini_statement(self):
        account_number = self.account["account_number"]
        account_transactions = [
            item for item in self.transactions if item.get("account") == account_number
```

```
        ][-10:]
        lines = [
            f"Account: {account_number}",
            f"Available balance: Rs. {self.account['balance']:.2f}",
            f"Pending cheques: Rs. {self.pending_cheque_total(account_number):.2f}",
            "",
            "Recent transactions:",
            "----------------------------------------",
        ]
        if not account_transactions:
            lines.append("No transactions available.")
        else:
            for item in account_transactions:
                lines.extend(
                    [
                        f"{self.display_time(item.get('date'))} | {item.get('type')}",
                        f"Amount: Rs. {float(item.get('amount', 0)):.2f} | "
                        f"Balance: Rs. {float(item.get('balance', 0)):.2f}",
                        f"Status: {item.get('status', 'Completed')} | Ref:
{item.get('reference', 'N/A')}",
                        "----------------------------------------",
                    ]
)
self.show_receipt("Mini Statement", lines)
def logout(self):
self.account = None
self.show_home()
def main():
```

```
root = tk.Tk()
ATMApp(root)
root.mainloop()
if __name__ == "__main__":
main();
```